

第3章 动态规划

提纲

CONTENTS

1 动态规划概述

5 字符串动态规划

2 一维动态规划

6 背包动态规划

3 二维动态规划

7 树形动态规划

4 三维动态规划

8 区间动态规划

1 动态规划概述

说明

动态规划将要解决的问题转化为一系列的子问题并且逐步加以解决，将前面解决子问题的结果作为后续解决子问题的条件，并且避免无意义的穷举。

动态规划求解问题的性质和步骤

采用动态规划求解的问题一般要具有以下3个性质

- ① **最优子结构**：如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构性质，即满足最优性原理。
- ② **无后效性**：如果某个阶段状态一旦确定，就不受以后决策的影响，也就是说某个状态以后的决策不会影响以前的状态。
- ③ **重叠子问题**：一个问题分解的若干子问题之间是不独立的，其中一些子问题在后面的决策中可能被多次重复使用。该性质并不是动态规划适用的必要条件，但是如果没有这条性质，动态规划算法同其他算法相比就不具备优势。

一般来说动态规划算法设计要经历以下几个步骤

- ① **确定状态**：将问题求解中各个阶段所处的各种情况用不同的状态表示出来。
- ② **确定状态转移方程**：描述求解中各个阶段的状态转移和指标函数的关系。
- ③ **确定初始条件和边界情况**：状态转移方程通常是一个递推式，初始条件通常指定递推的起点，在递推中需要考虑一些特殊情况，称为边界情况。
- ④ **确定计算顺序**：也就是指定求状态转移方程的顺序，是顺序求解还是逆序求解。
- ⑤ **消除冗余**：如采用滚动数组进一步提高时空性能。

2 一维动态规划

说明

一维动态规划是指设计动态规划算法中采用一维动态规划数组，也称为线性动态规划。

最大连续子序列和

- 给定一个含 n ($n \geq 1$) 个整数的序列，要求求出其中最大连续子序列的和。
- 序列 $(-2, 11, -4, 13, -5, -2)$ 的最大子序列和为20。
- 序列 $(-6, 2, 4, -7, 5, 3, 2, -1, 6, -9, 10, -2)$ 的最大子序列和为16。
- 规定一个序列最大连续子序列和至少是0，如果小于0，其结果为0。

1,-5, 3,4,-2,-3,10

以 **1** 开始的子数组:

1↓

1,-5↓

1,-5,3↓

1,-5,3,4↓

1,-5,3,4,-2↓

1,-5,3,4,-2,-3↓

1,-5,3,4,-2,-3,10↓

以 **-5** 开始的子数组:

-5↓

-5,3↓

-5,3,4↓

-5,3,4,-2↓

-5,3,4,-2,-3↓

-5,3,4,-2,-3,10↓

以 **3** 开始的子数组: 略...↓

以 **4** 开始的子数组: 略...↓

以 **-2** 开始的子数组: 略...↓

以 **-3** 开始的子数组: 略...↓

以 **10** 开始的子数组: 略...↓

枚举后再对所有的子数组求和, 可以得到如下代码

简单算法：

```
int MaxSum(int n, int *a, int & besti, int & bestj)
{
    int sum=0;
    for (int i=1;i<=n;i++) {
        for (int j=i;j<=n;j++) {
            int thissum=0;
            for (int k=i;k<=j;k++) thissum+=a[k];
            if (thissum>sum) {
                sum=thissum;
                besti=i;
                bestj=j;
            }
        }
    }
    return sum;
}
```

a1
a1 a2
a1 a2 a3
a1 a2 a3 a4
a1 a2 a3 a4 a5

a2
a2 a3
a2 a3 a4
a2 a3 a4 a5

a3
a3 a4
a3 a4 a5

从代码中可以看出，该算法是 $O(N^3)$ 的复杂度。

枚举子数组的起始点 i 与结束点 j 为 $O(N^2)$ ，
求子数组的和为 $O(N)$ ，最终复杂度为 $O(N^3)$ 。

- 最直观的解法往往不是最优秀的解法，从代码中可以看出，我们在求子数组和的时候重复做了很多事，即红色的代码，这段明显可以删去。
- 原因是当求以j为结尾的子数组的和时，以j-1为结尾的子数组的和已经求得，所以 $\text{sum}(j)$ 就是 $\text{sum}(j-1)+a[j]$ （ $\text{sum}(j)$ 代表以j为结尾的子数组的和）。

```
int MaxSum(int n, int *a, int & besti, int & bestj)
{
    int sum=0;
    for (int i=1;i<=n;i++) {
        for (int j=i;j<=n;j++) {
            int thissum=0;
            for (int k=i;k<=j;k++) thissum+=a[k];
            if (thissum>sum) {
                sum=thissum;
                besti=i;
                bestj=j;
            }
        }
    }
    return sum;
}
```

a1
a1 a2
a1 a2 a3
a1 a2 a3 a4
a1 a2 a3 a4 a5

解法2：技巧改进

```
int MaxSum(int n, int *a, int & besti, int & bestj)
{
    int sum=0;
    for (int i=1;i<=n;i++) {
        for (int j=i;j<=n;j++) {
            int thissum=0;
            for (int k=i;k<=j;k++) thissum+=a[k];
            if (thissum>sum) {
                sum=thissum;
                besti=i;
                bestj=j;
            }
        }
    }
    return sum;
}
```

a1
a1 a2
a1 a2 a3
a1 a2 a3 a4
a1 a2 a3 a4 a5

```
int MaxSum(int n, int *a,
int & besti, int & bestj) {
    int sum=0;
    for (int i=1;i<=n;i++) {
        int thissum=0;
        for (int j=i;j<=n;j++) {
            thissum+=a[j];
            if (thissum>sum) {
                sum=thissum;
                besti=i;
                bestj=j;
            }
        }
    }
    return sum;
}
```

$$\sum_{k=i}^j a_k = a_j + \sum_{k=i}^{j-1} a_k$$

- 最直观的解法往往不是最优秀的解法，从代码中可以看出，我们在求子数组和的时候重复做了很多事，即红色的代码，这段明显可以删去。
- 原因是当求以j为结尾的子数组的和时，以j-1为结尾的子数组的和已经求得，所以sum(j)就是sum(j-1)+a[j]（sum(j)代表以j为结尾的子数组的和）。

从例子中观察规律



我们可以想到，当枚举起始点 i 与结束点 j 的时候，这里的复杂度就是 $O(N^2)$ 。

那我们可不可以只枚举起始点 i ，或者只枚举结束点 j 而最终得到最大子数组的和呢？

假如现在我们只枚举数组的结束点 j 。

假如现在我们只枚举数组的结束点j

1, -5, 3, 4, -2, -3, 10

以下标 **0** 为结束的数组: **1**↓

以下标 **1** 为结束的数组: **1, -5**↓

以下标 **2** 为结束的数组: **1, -5, 3**↓

以下标 **3** 为结束的数组: **1, -5, 3, 4**↓

以下标 **4** 为结束的数组: **1, -5, 3, 4, -2**↓

以下标 **5** 为结束的数组: **1, -5, 3, 4, -2, -3**↓

以下标 **6** 为结束的数组: **1, -5, 3, 4, -2, -3, 10**↓

观察枚举的过程：

- 数组 $dp[j]$ ：表示以下标 j 结束的最大子数组的和的值。
- 以下标0为结束的数组的最大子数组和就是该独立的元素1。即 $dp[0]=1$ 。
- 思考？以下标1为结束的数组：1,-5的最大子数组和是什么？
- 注意：这里指的最大子数组必须以下标1(或者说 j)结束

以下标 0 为结束的数组：1↓

以下标 1 为结束的数组：1,-5↓

以下标 2 为结束的数组：1,-5,3↓

以下标 3 为结束的数组：1,-5,3,4↓

以下标 4 为结束的数组：1,-5,3,4,-2↓

以下标 5 为结束的数组：1,-5,3,4,-2,-3↓

以下标 6 为结束的数组：1,-5,3,4,-2,-3,10↓

观察以上枚举的过程：

- 思考？

- 以下标1为结束的数组：1,-5的最大子数组和是什么？

- 答案很明确：

- 如果以下标0结束的的最大子数组的和为正数，那么 $dp[1]$ 就是 $dp[0]+a[1]$ ；

- 如果如果以下标0结束的的最大子数组的和为负数，那么 $dp[1]$ 就是 $a[1]$ 。

所以 $dp[1]=dp[0]+a[1]=-4$ 。

- 以下标2为结束的数组：

1,-5,3的最大子数组和是什么？

以下标 0 为结束的数组：1↓

以下标 1 为结束的数组：1,-5↓

以下标 2 为结束的数组：1,-5,3↓

以下标 3 为结束的数组：1,-5,3,4↓

以下标 4 为结束的数组：1,-5,3,4,-2↓

以下标 5 为结束的数组：1,-5,3,4,-2,-3↓

以下标 6 为结束的数组：1,-5,3,4,-2,-3,10↓

观察以上枚举的过程：

- 思考？

- 以下标2为结束的数组：1,-5,3的最大子数组和是什么？

- 答案很明确：

- 如果 $dp[1] > 0$ ，则 $dp[2] = dp[1] + a[2]$ 。

否则 $dp[2] = a[2]$ 。

所以 $dp[2] = 3$ 。

以下标 **0** 为结束的数组： **1**↓

以下标 **1** 为结束的数组： **1,-5**↓

以下标 **2** 为结束的数组： **1,-5,3**↓

以下标 **3** 为结束的数组： **1,-5,3,4**↓

以下标 **4** 为结束的数组： **1,-5,3,4,-2**↓

以下标 **5** 为结束的数组： **1,-5,3,4,-2,-3**↓

以下标 **6** 为结束的数组： **1,-5,3,4,-2,-3,10**↓

解

设计一维动态规划 dp ， $dp[i]$ ($1 \leq i \leq n-1$) 表示以元素 a_i 结尾的最大连续子序列和，显然 $dp[i-1]$ 表示以元素 a_{i-1} 结尾的最大连续子序列和。判断 a_i 分为两种情况：

- ① 将 a_i 合并到前面以元素 a_{i-1} 结尾的最大连续子序列中，此时有 $dp[i] = dp[i-1] + a_i$ 。
- ② 不将 a_i 合并到前面以元素 a_{i-1} 结尾的最大连续子序列中，即从 a_i 开始构造一个连续子序列，此时有 $dp[i] = a_i$ 。

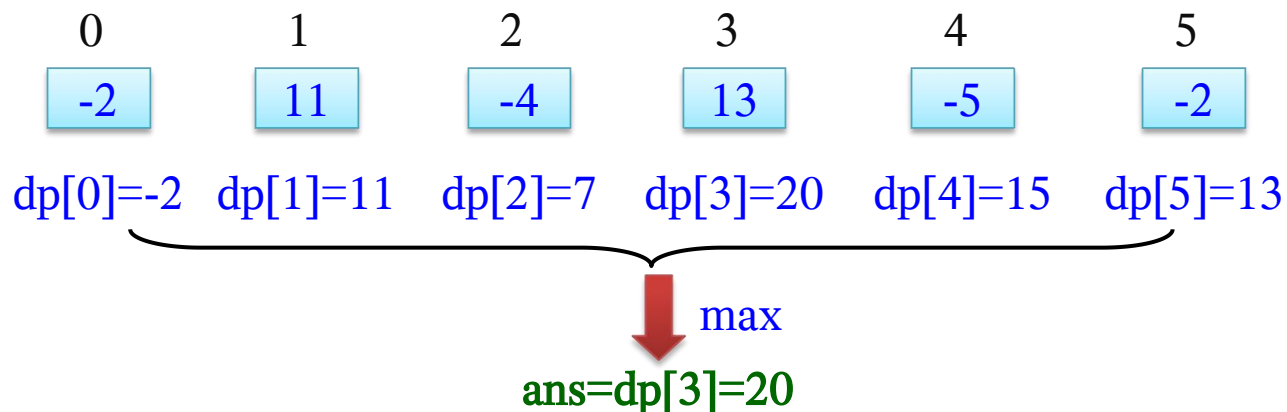


$$dp[0] = a[0]$$

初始条件

$$dp[i] = \max(dp[i-1] + a_i, a_i)$$

$i > 0$



- 求出 dp 中的最大元素 ans 。
- 由于本题中最大连续子序列和至少为0（或者说最大连续子序列可以为空序列），所以最后的最大连续子序列和应该为 $\max(ans, 0)$ 。

```

int dp[MAXN];
int maxSubSum(vector<int> & a)           //求最大连续子序列和
{
    int n=a.size();
    memset(dp,0,sizeof(dp));

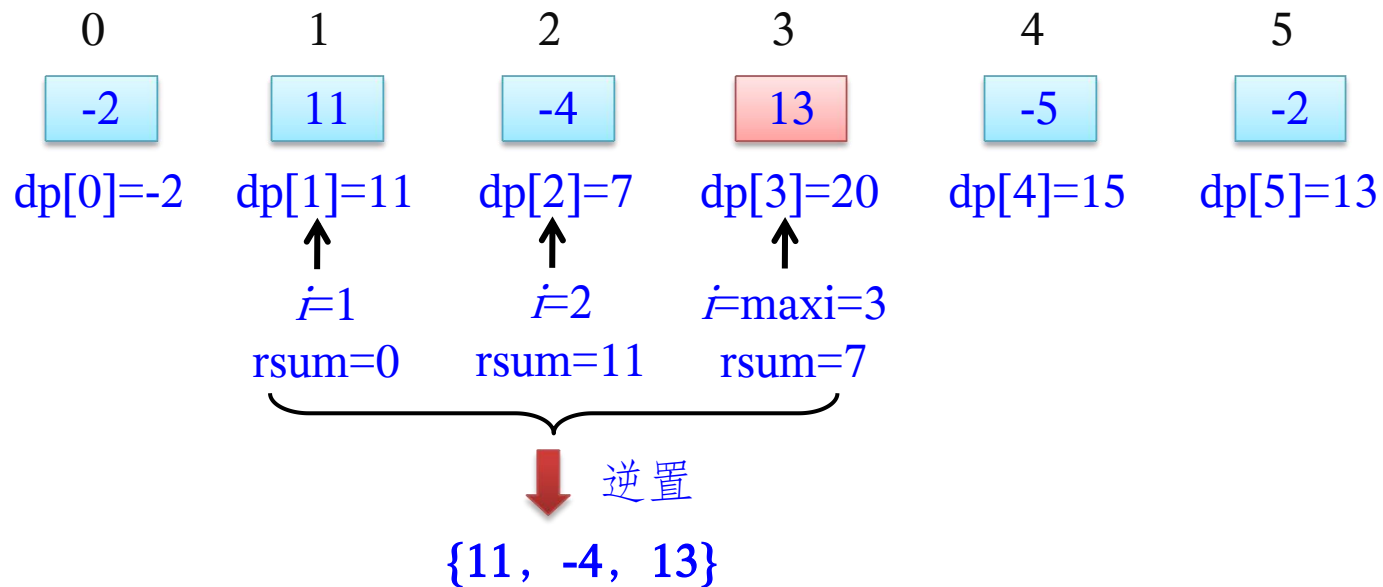
    dp[0]=a[0];
    for(int i=1;i<n;i++)                 //计算顺序是j从1到n-1
        dp[i]=max(dp[i-1]+a[i],a[i]);

    int ans=dp[0];
    for(int i=1;i<n;i++)
        ans=max(ans,dp[i]);
    return max(ans,0);
}

```

【算法分析】 maxSubSum算法含两个for循环（实际上第二个for循环可以合并到第一个for循环中），对应时间复杂度均为 $O(n)$ 。算法中应用了dp数组，对应的空间复杂度为 $O(n)$ 。

- 当求出 dp 后可以推导出一个最大连续子序列（实际上这样的最大连续子序列可能有多个，这里仅仅求出其中的一个）。
- 先在 dp 数组中求出最大元素的序号 $maxi$ ， i 从 $maxi$ 序号开始在 a 中向前查找， $rsum$ 从 $dp[maxi]$ 开始递减 $a[i]$ ，直到 $rsum$ 为0，对应的 a 中子序列就是一个最大连续子序列。



```
vector<int> maxSub(vector<int>& a)    //求一个最大连续子序列
```

```
{   int n=a.size();
```

```
    vector<int> x;
```

```
    int maxi=0;
```

```
    for(int i=1;i<n;i++)
```

```
    //求dp中最大元素序号maxi
```

```
        if(dp[i]>dp[maxi])
```

```
            maxi=i;
```

```
    int rsum=dp[maxi];
```

```
    int i=maxi;
```

```
    while(i>=0 && rsum!=0)
```

```
    {   rsum-=a[i];
```

```
        x.push_back(a[i]);
```

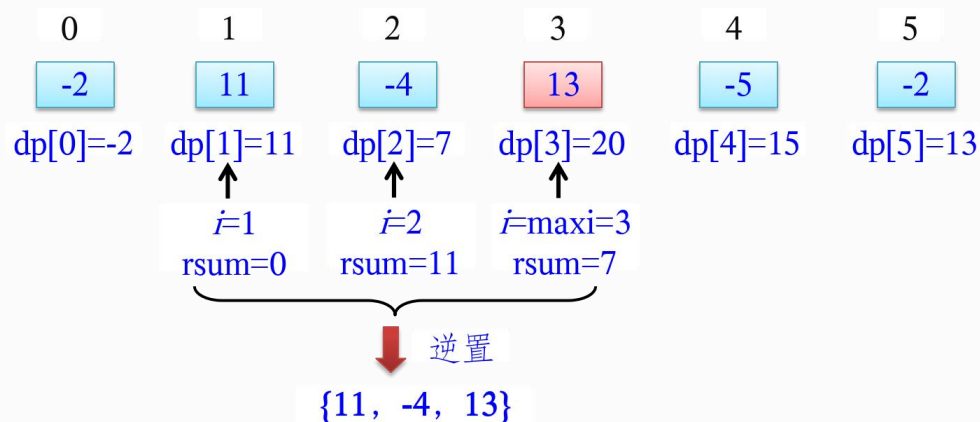
```
        i--;
```

```
    }
```

```
    reverse(x.begin(),x.end());
```

```
    return x;
```

```
}
```



空间优化

- 如果只需要求最大连续子序列和，可以采用滚动数组优化空间。
- `maxSubSum`算法中用 j 标识阶段，由于`dp[j]`仅仅与`dp[j-1]`相关，将一维`dp`数组改为单个变量`dp`。

```
int maxSubSum1(vector<int>& nums)    //求最大连续子序列和
{
    int n=nums.size();
    int dp;
    dp=nums[0];
    int ans=dp;
    for(int i=1;i<n;i++)              //计算顺序是i从1到n-1循环
    {
        dp=max(dp+nums[i],nums[i]);
        ans=max(ans,dp);
    }
    return max(ans,0);
}
```

空间复杂度为 $O(1)$

还有新思路吗？

1, -5, 3, 4, -2, -3, 10

那么最大子数组和即为 12. 由 3, 4, -2, -3, 10 组成.

我们考虑以下标 i 为结尾的数组:

以下标 0 为结束的数组: 1

以下标 1 为结束的数组: 1, -5

以下标 2 为结束的数组: 1, -5, 3

以下标 3 为结束的数组: 1, -5, 3, 4

以下标 4 为结束的数组: 1, -5, 3, 4, -2

以下标 5 为结束的数组: 1, -5, 3, 4, -2, -3

以下标 6 为结束的数组: 1, -5, 3, 4, -2, -3, 10

不信？ 换个例子试试？

```

int getmax3_N(int a[],int n)↓
{↓
    int sum=0,sumMin=0,Max=a[0];
    for(int i=0;i<n;i++)↓
    {↓
        if(sumMin>sum)↓
            sumMin=sum;↓
        sum=sum+a[i];↓
        if(Max<sum-sumMin)↓
            Max=sum-sumMin;↓
    }↓
    return Max;↓
}

```

扩展1：如何求最大子数组的位置？

- 如何求最大子数组的起点下标、结束点下标
- 求最大子数组的位置时，可能会出现一个数组中有许多和是一样的最大子数组，这里我们可以限定，优先考虑位置最靠前（起点靠前）的最大子数组，当最大子数组的起点也相同时，我们选定结束点靠前的。
- 例如：求2,3,-1,1,-5,4,1的最大子数组的和，并求该子数组的位置：
- 结果为：5 0 1。
- 这里第一个整数5代表最大子数组的和，0代表最大子数组的起点下标，1代表最大子数组的结束点下标。虽然5 5 6也对，但是子数组的起点靠后，5 0 3也对，但是子数组的结束点靠后。

起始点与结束点下标如何来记录？

- 由于起始点下标、结束点下标都是靠前的子数组，所以我们在动态规划的时候最好**从后向前递推**
- $dp[i]$ 表示的值就是以**下标 i 为开始的**最大子数组的值，
- 当 $dp[i]$ 与 $dp[j]$ 相同时，选取 i, j 中较小的下标作为**起点**。
- 当出现**2,3,-1,1**这样的情况时，需要选择**3**的下标作为**结束点**。

```
if(dp[i+1]>0)
{
    dp[i]=dp[i+1]+a[i];
    tmp[i]=tmp[i+1];
}
else
{
    dp[i]=a[i];
    tmp[i]=i;
}
```

- 这里tmp变量记录子数组的结束点,
- 当dp[i+1]==0时, 我们要选取i作为此时的子数组结束点。
- 当2,3,-1,1; 当我们计算dp[1]时, 它的结束点就是1, 而不是dp[2]的结束点3。

最终最大的dp[i]代表最大子数组的和,
最小的i代表子数组的起始点,
tmp[i]代表子数组的结束点。

```

void getMax(int a[],int n)
{
    int dp,tmp;
    dp=Max=a[n-1];
    tmp=n-1;
    B=n-1;
    E=n-1;
    for(int i=n-2;i>=0;i--)
    {
        if(dp>0)
            dp=dp+a[i];
        else
        {
            dp=a[i];
            tmp=i;
        }
        if(Max<=dp)
        {
            B=i;
            E=tmp;
            Max=dp;
        }
    }
}

```

int B,E,Max;

- **B**为子数组的起始点下标
- **E**为子数组的结束点下标
- **Max**为最大子数组的和
- **tmp**记录子数组的结束点
- getMax求数组a(元素个数为n,以下标0作为起始点)的最大子数组,外加起点下标与结束点下标。

由于dp[i]只与dp[i+1]有关系, tmp[i]只与tmp[i+1]有关系, 所以我们可以省略数组, 改为变量dp,tmp。

更上一层楼

1.将问题拓展到**两段和**，一个数组的两段最大子数组的和。

例如1,-2,2,3,-4结果就是6。第一段就是1，第二段就是2,3。

2.将问题拓展到一个循环数组，就是一个圈，下标为 $n-1$ 的元素后面的元素下标为0，求这样的数组的最大子数组的和。

3.加入限制条件 m ， m 代表子数组的最大长度，即求长度不超过 m 的最大子数组的和。

4.将问题拓展到二维，求一个矩阵的最大子矩阵的和

(2) 最大m子段和问题

问题描述：给定由n个整数(可能为负整数)组成的序列 $a_1, a_2, \dots, a_n$, 以及一个正整数m, 要求确定序列的m个不相交子段, 使这m个子段的总和达到最大。

解题思路：设 $b(i, j)$ 表示数组a的前j项中i个子段和的最大值, 且第i个子段含 $a[j]$ ($1 \leq i \leq m, 1 \leq j \leq n$)。则所求的最优值显然为

$$\max_{1 \leq j \leq n} b(m, j)$$

与最大子段和问题类似, 计算 $b(i, j)$ 的递归式为

$$b(i, j) = \max \{b(i, j-1) + a[j], \max_{i-1 \leq t < j} b(i-1, t) + a[j]\} \quad (1 \leq i \leq m, 1 \leq j \leq n)$$

初始时, $b(0, j)=0, (1 \leq j \leq n); b(i, 0)=0, (1 \leq i \leq m)$ 。

简单点：
求序列的最大两个子段和



求解：
一个数组的两段最大子数组的和

- 例如：
 - 1,-2,2,3,-4结果就是6。第一段就是1，第二段就是2,3。
- 已知整数序列(数组) $A=\{a_1, a_2, \dots, a_n\}$, 定义函数 $d(A)$ 如：

$$d(A) = \max_{1 \leq s_1 \leq t_1 < s_2 \leq t_2 \leq n} \left\{ \sum_{i=s_1}^{t_1} a_i + \sum_{j=s_2}^{t_2} a_j \right\}$$

算法分析：

1	-1	2	2	3	-3	4	-4	5	-5
---	----	---	---	---	----	---	----	---	----

设：

$dp1[i]$ 代表以下标 i 结束的最大子数组的值，

$max1[i]$ 代表 $dp1[0] \sim dp1[i]$ 最大的那个值；

设

$dp2[i]$ 代表以下标 i 开始的最大子数组的值，

$max2[i]$ 代表 $dp1[i] \sim dp1[n-1]$ 中最大的那个值。

例如， $max1[6]$ 就是 $dp[6]$ 即 $\{2, 2, 3, -3, 4\}$ ； $max2[7]$ 就是 $dp[8]$ 即 $\{5\}$ ，

所以上面那个例子求出的解就是 $max1[6] + max2[7]$ 。

为何是 $max1[6] + max2[7]$ ，是因为在 $max1[i] + max2[i+1]$ ($i=0 \sim n-2$) 中，它最大。

只需要枚举这两段的分界点 i ， $0 \leq i \leq n-2$ ，就可以了。


```

int max1[MAXN], max2[MAXN];
int getMax(int a[], int n)
{
    int Max, dp;

    max1[0] = dp = a[0];
    for(int i=1; i<n; i++)
    {
        if(dp<0)
            dp = a[i];
        else
            dp = dp + a[i];
        if(max1[i-1]<dp)
            max1[i] = dp;
        else
            max1[i] = max1[i-1];
    }

    max2[n-1] = dp = a[n-1];
    for(int i=n-2; i>=0; i--)
    {
        if(dp<0)
            dp = a[i];
        else
            dp = dp + a[i];
        if(max2[i+1]<dp)
            max2[i] = dp;
        else
            max2[i] = max2[i+1];
    }

    Max = max1[0] + max2[1];
    for(int i=1; i<n-1; i++)
        if(Max<max1[i] + max2[i+1])
            Max = max1[i] + max2[i+1];
    return Max;
}

```

利用标准动态规划求最大子子段和的方法，求出dp1[]、dp2[]、max1[]、max2[]；枚举出max1[i]+max2[i+1](i=0~n-2)中，最大的那个数，就是结果。

实战一最大子序和 (LeetCode53)

问题描述:给定一个含 n ($1 \leq n \leq 30000$) 个整数的数组 `nums` (整数值在-100000到100000之间), 找到一个具有最大和的连续子数组 (子数组最少包含一个元素), 返回其最大和。

例如`nums={-2,-1}`, 结果为-1。

要求设计如下函数:

```
class Solution {  
public:  
    int maxSubArray(vector<int>& nums) { }  
};
```

解

- 本题采用动态规划求解的原理见8.2.1节，这里仅仅需要求最大连续子序列和，而且该最大连续子序列至少含一个元素。
- 采用滚动数组。

```
class Solution {
public:
    int maxSubArray(vector<int>& nums)
    {
        int n=nums.size();
        if(n==1) return nums[0];
        int dp;
        dp=nums[0];
        int ans=dp;
        for(int j=1;j<n;j++)
        {
            dp=max(dp+nums[j],nums[j]);
            ans=max(ans,dp);
        }
        return ans;
    }
};
```

//不能改为max(ans,0)



三省吾身

问题求解—自下而上

自下而上算法空间优化